Kigali Shaolin Temple Management System

Frontend Website, Admin Portal (CMS), and Backend API

Version 1.1.0

Purpose:
A unified professional blueprint combining system analysis, UI/UX design, CMS control, and backend logic.

Prepared for: Technical Documentation
Year: 2025

Contents

1 General Project Overview **3**

2 System Architecture **3**

3 Branding and UI Design System (Tailwind) **4**
3.1 Color Palette 4
3.2 Tailwind Configuration 4
3.3 Design Aesthetics (Premium Matrix) 4
3.4 Typography 5

4 Frontend Website (Public) **5**
4.1 Design Goals 5

5 Frontend Pages and Sections **5**
5.1 Header and Navigation 5
5.2 Hero Section 5
5.3 About Us Section 6
5.4 Programs and Training 6
5.5 Blogs 6
5.6 Clubs 7
5.7 Contact Us 7
5.8 Gallery 7

6 Admin Portal (CMS Dashboard) **7**
6.1 Authentication and Roles 7
6.2 Dashboard Modules 8

7 Backend (API and Logic) **8**
7.1 Core Responsibilities 8
7.2 Technology Stack 8
7.3 API Interceptors and Security 9
7.4 Media Upload Architecture 9

8 Database Models **10**

9 Advanced System Features **10**
9.1 Audit Logging 10
9.2 Data Recovery and Soft Deletes 10
9.3 Interactive API Documentation 11
9.4 ID Obfuscation (UUID Encryption) 11

9.5 Database Schema Standardization 11
9.6 Frontend Stability Enhancements 12

10 Docker Infrastructure and Deployment 12
10.1 Container Services 12
10.2 Deployment Workflow 12
10.3 Production Build Commands 13
10.4 SPA Routing and Nginx 13
10.5 Automated Initialization 13

11 Production Hosting (Render.com) 13
11.1 Render Blueprint (render.yaml) 13
11.2 Production Data Safety 14

12 Resolved Technical Issues (v1.1.0) 15

13 Non-Functional Requirements 15

14 Final Summary 15

1 General Project Overview

Project Name: Kigali Shaolin Temple Management System

Purpose: To promote Shaolin culture in Rwanda, manage clubs, programs, blogs, and allow administrators to dynamically control website content through a secure Content Management System (CMS).

This document serves as a master reference for:

- System development
- Technical documentation
- Project proposal or defense
- Team onboarding

2 System Architecture

The system follows a modern, scalable, production-ready architecture.

```
[ Frontend (React + Vite) ]
            |
         REST API
            |
[ Backend (Node.js + Express) ]
            |
[ Admin Portal (CMS Dashboard) ]
            |
[ Database (PostgreSQL + Sequelize) ]
```

Architecture Description

- Frontend: Public-facing website for visitors
- Admin Portal: Secure CMS for content management
- Backend API: Business logic, authentication, and data access
- Database: Persistent storage

3 Branding and UI Design System (Tailwind)

3.1 Color Palette

Role	Color
Primary	#7B1E1E (Shaolin Red)
Dark	#111111
Accent	#C9A24D (Heritage Gold)
Text	#FFFFFF
Muted	#9CA3AF

3.2 Tailwind Configuration

```
extend: {
  colors: {
    primary: '#7B1E1E',
    dark: '#0A0A0A',
    gold: '#C4A747',
  }
}
```

3.3 Design Aesthetics (Premium Matrix)

The system employs a high-end visual language referred to as the "Premium Matrix" aesthetic:

- **Glassmorphism**: Interactive cards use `bg-white/70 backdrop-blur-xl` for a modern, layered feel.
- **Cinematic Overlays**: Hero sections and media displays use deep gradients and subtle glows.
- **Premium Shadows**: Custom shadows (`shadow-premium`) provide depth without clutter.
- **Interactive Micro-animations**: Smooth hover effects and transitions are applied globally.

3.4 Typography

Usage	Font
Headings	Playfair Display (Serif)
Body Text	Inter / Poppins (Sans-serif)

4 Frontend Website (Public)

4.1 Design Goals

- Cinematic visuals
- Martial arts strength and discipline
- Traditional yet modern appearance
- Fully CMS-driven content
- Mobile-first responsiveness

5 Frontend Pages and Sections

5.1 Header and Navigation

- Logo
- Navigation: Home, About, Blog, Clubs, Contact
- Language switcher (RW / EN / FR)

CMS Control: None (static navigation)

5.2 Hero Section

- Full-screen background image
- Dark overlay
- Title, subtitle, CTA button
- Carousel slider

CMS Controlled:

- Title and subtitle

- Background images
- Button text and link

5.3 About Us Section

- Image and text layout
- Founder quotation

CMS Controlled:

- About text
- Founder name
- Images
- Year founded

5.4 Programs and Training

- Online Training
- In-Person Training
- Global Retreats

Each program contains:

- Image
- Title
- Description
- Status (active or inactive)

5.5 Blogs

- Featured blog
- Blog listing
- Blog detail page

5.6 Clubs

- Club name
- Location
- Instructor
- Description
- Images

5.7 Contact Us

- Name
- Email
- Message

5.8 Gallery

- Photo and video media
- Categorization by event
- Date and title metadata

Backend Requirements:

- Message storage
- Admin replies
- Email notifications

6 Admin Portal (CMS Dashboard)

6.1 Authentication and Roles

Role	Description
Super Admin	Complete system control and audit access
Content Manager	Global content management across all modules
Blogger	Focused access: creation and management of own blogs
Editor	Standard content editing and review

6.2 Dashboard Modules

- Dashboard overview and system statistics
- Homepage (Hero Slides) manager
- About manager
- Programs manager
- Blog manager
- Clubs and Tutorials manager
- Gallery (Media) manager
- Messages manager
- Audit Logs (Activity tracking)
- Language manager
- Users and roles

7 Backend (API and Logic)

7.1 Core Responsibilities

- Authentication and authorization using JWT
- Content APIs
- Image uploads
- Email services
- Role-based access control

7.2 Technology Stack

Backend:

- Node.js with Express
- JWT authentication and Authorization middleware
- Multer and Cloudinary for media storage

- Nodemailer for email invitations and password resets
- Swagger (OpenAPI) for interactive API documentation
- Audit Logging for administrative actions
- **API Resilience**: Centralized `apiClient.js` implementation

7.3 API Interceptors and Security

The frontend communication layer is hardened with:

- **Request Interceptors**: Automatic injection of JWT Bearer tokens from local storage.
- **Response Interceptors**: Global handling of 401 Unauthorized errors with automatic cleanup and redirect to login.
- **Centralized Configuration**: Environment-based Base URL management for seamless dev/prod switching.

7.4 Media Upload Architecture

The system supports two upload modes controlled by the `UPLOAD_MODE` environment variable:

Mode	Storage	Persistence
`local`	Server filesystem (`/uploads`)	Ephemeral (lost on restart)
`cloud`	Cloudinary CDN	Permanent

Production Requirement: Always use `UPLOAD_MODE=cloud` with valid Cloudinary credentials on Render (and any platform with ephemeral storage), as local files are wiped on container restart.

Cloudinary is configured once at module load in `uploadUtils.js`:

```
cloudinary.config({
  cloud_name: process.env.CLOUD_NAME,
  api_key: process.env.API_KEY,
  api_secret: process.env.API_SECRET,
});
```

8 Database Models

```
User(id, name, email, password, role, image, reset_token,
   created_at)

HeroSlide(id, title, subtitle, image, button_text, button_link)

About(id, content, founder_name, image)

Program(id, name, description, image, status)

Blog(id, title, content, image, category, published, created_at)

Club(id, name, description, location, image)

ClubTutorial(id, club_id, title, video_url)

Gallery(id, title, description, url, media_type, event_date,
   category)

ContactMessage(id, name, email, message, read, created_at)

AuditLog(id, user_id, action, entity, entity_id, details,
   ip_address)
```

9 Advanced System Features

9.1 Audit Logging

All administrative actions (Create, Update, Delete, Restore) are logged in the system. Each log entry captures:

- The administrator who performed the action
- The type of action and the entity affected
- A timestamp and the administrator's IP address

9.2 Data Recovery and Soft Deletes

The system implements soft deletes (paranoid mode), allowing administrators to restore accidentally deleted content. Permanent deletion is only possible through a deliberate

"Force Delete" action.

9.3 Interactive API Documentation

The backend includes a Swagger (OpenAPI) interface, providing real-time documentation and a testing environment for all API endpoints.

9.4 ID Obfuscation (UUID Encryption)

To enhance security and prevent exposure of internal database structure, the system implements ID obfuscation using Base64URL encoding:

- All public-facing UUIDs are encoded using Base64URL (URL-safe encryption)
- Backend transparently decodes obfuscated IDs via middleware
- Prevents enumeration attacks and information disclosure
- Maintains backward compatibility with existing UUID structure

Implementation:

```
// Encoding: UUID -> Base64URL
const encodeUUID = (uuid) => {
  const hex = uuid.replace(/-/g, '');
  const buffer = Buffer.from(hex, 'hex');
  return buffer.toString('base64')
    .replace(/\+/g, '-')
    .replace(/\//g, '_')
    .replace(/=/g, '');
};

// Decoding middleware applied to all routes
router.get('/:id', decodeParam('id'), controller.getById);
```

9.5 Database Schema Standardization

All Sequelize models have been standardized with explicit $snake_case$ timestamp mappings to ensure database consistency:

- Explicit mapping: `createdAt:` '$created_at$', `updatedAt:` '$updated_at$'
- Resolves Sequelize ordering and query inconsistencies

- Prevents 500 Internal Server Errors from column name mismatches
- Applied to all models: Club, Program, Gallery, About, HeroSlide, User, Blog, ContactMessage, AuditLog

9.6 Frontend Stability Enhancements

The admin portal includes robust error handling and defensive programming practices:

- Optional chaining for safe property access (`pagination?.totalItems`)
- Default values for undefined data states
- Pagination protection against crashes during network latency
- Error boundaries for graceful degradation

10 Docker Infrastructure and Deployment

The system is fully containerized using Docker, enabling consistent deployment and simplified environment management across development, testing, and production.

10.1 Container Services

The ecosystem consists of three orchestrated services. Production images are published on Docker Hub under the `andremugabo` registry with the `:prod` tag:

- **kst-backend**: `andremugabo/kst-backend:prod` — Node.js API with Cloudinary integration and custom entrypoint for DB synchronization and seeding.
- **kst-frontend**: `andremugabo/kst-frontend:prod` — Production-optimized public website served via Nginx.
- **kst-admin**: `andremugabo/kst-admin:prod` — Production-optimized CMS portal served via Nginx.

10.2 Deployment Workflow

The entire stack can be initialized with a single command:

```
docker compose up -d --build
```

10.3 Production Build Commands

For production environments, the Frontend and Admin apps are built with the live API URL **and upload URL** baked in at build time (Vite resolves `import.meta.env` at build time, not runtime):

```
cd kst.rw
# Build and push all images (uses push-images.sh)
bash push-images.sh

# Manual example for Frontend:
docker build \
  --build-arg VITE_API_URL=https://kst-backend.onrender.com/api \
  --build-arg VITE_UPLOAD_URL=https://kst-backend.onrender.com \
  -t andremugabo/kst-frontend:prod ./kst-frontend
docker push andremugabo/kst-frontend:prod
```

10.4 SPA Routing and Nginx

Both the Admin Portal and Frontend are configured with a custom Nginx configuration to support Single Page Application (SPA) routing, ensuring that all deep-linked URLs correctly resolve to `index.html`.

10.5 Automated Initialization

The backend container utilizes a dedicated `docker-entrypoint.sh` script that:

1. Waits for the database service to be healthy.
2. Runs `npm run db:reset` to synchronize models and seed initial administrative users.
3. Starts the production Express server.

11 Production Hosting (Render.com)

For production environments, the system is designed to be hosted on Render.com using their Infrastructure-as-Code (Blueprint) feature.

11.1 Render Blueprint (render.yaml)

A `render.yaml` file is provided, which provisions all three web services using Docker images. Key environment variables for the backend:

Variable	Purpose
`NODE_ENV`	Set to `production`
`DATABASE_URL`	Neon.tech PostgreSQL connection string
`BACKEND_URL`	Public backend URL for file path construction
`UPLOAD_MODE`	`cloud` (required for persistence)
`CLOUD_NAME`	Cloudinary cloud name
`API_KEY`	Cloudinary API key
`API_SECRET`	Cloudinary API secret
`JWT_SECRET`	Auto-generated by Render

Key environment variables for the frontend and admin:

Variable	Purpose
`VITE_API_URL`	Backend API base URL
`VITE_UPLOAD_URL`	Backend base URL for media resolution

Note: `VITE_UPLOAD_URL` must also be passed as a Docker `-build-arg` since Vite bakes environment variables at build time.

11.2 Production Data Safety

The system implements environment-aware startup logic. When `NODE_ENV=production` is detected:

- Full database resets (`db:reset`) are disabled.
- Safe synchronization (`db:sync`) is performed using Sequelize's `alter: true` mode.
- The seeder (`seed.js`) runs in upsert mode — it creates records if absent, and updates stale placeholder URLs if they already exist.

12 Resolved Technical Issues (v1.1.0)

Issue	Resolution
`ERR_NAME_NOT_RESOLVED` for placeholder images	Replaced `via.placeholder.com` with `placehold.co` in seed data and all components
Images disappear on page refresh	Root cause: `UPLOAD_MODE=cloud` with placeholder Cloudinary credentials silently fell back to ephemeral local storage. Fixed by calling `cloudinary.config()` at module load and providing real credentials.
Backend fails to start on Render	Removed hardcoded `PORT: 3005` from `render.yaml`; Render assigns ports dynamically.
All API requests blocked by CORS	Service name mismatch between Render hostnames and `allowedOrigins` in `server.js`. Fixed by correcting the CORS list.
Malformed image URLs	Fixed leading/trailing slash handling in `getMediaPath` in `apiClient.js`.
No image URLs in production frontend	`VITE_UPLOAD_URL` was missing as a Docker `build-arg`; added to both Dockerfiles and `push-images.sh`.

13 Non-Functional Requirements

- Mobile-first responsive design
- Search engine optimization
- Secure admin routes
- Optimized images
- Fast loading performance

14 Final Summary

The Kigali Shaolin Temple Management System consists of three integrated layers:

Layer	Role	Live URL
Frontend	Public Website	kst-frontend.onrender.com
Admin Portal	CMS Dashboard	kst-admin.onrender.com
Backend	API Engine	kst-backend.onrender.com

This design:

- Covers the majority of UI requirements from the provided design
- Uses Tailwind CSS for a modern and professional interface
- Enables scalability through CMS control